

BƯỚC 1: TẢI VÀ KHẢO SÁT SƠ BỘ DỮ LIỆU

```
1 # Doc file CSV
2 raw_data <- read.csv("All_GPUs.csv", stringsAsFactors = FALSE)
3
4 # Kiem tra du lieu
5 str(raw_data)
6 summary(raw_data)
7 colSums(is.na(raw_data))
```

read.csv(..., stringsAsFactors = FALSE): Lệnh này giúp R đọc file CSV nhưng giữ nguyên các cột văn bản dưới dạng chuỗi (character) thay vì tự động chuyển thành nhóm (Factor).

str(raw_data): Hiển thị cấu trúc của 3406 dòng và 34 cột.

colSums(is.na(raw_data)): Đây là lệnh cực kỳ quan trọng để đếm số giá trị khuyết (NA). Nó cung cấp bằng chứng thép cho việc tại sao chúng ta phải xử lý dữ liệu ở các bước sau.

Kết quả trên console:

```
1 > colSums(is.na(raw_data))
2      Architecture      Best_Resolution      Boost_Clock
3      Core_Speed      DVI_Connection
4      0      0      0
5      0      750
6      Dedicated      Direct_X      DisplayPort_Connection
7      HDMI_Connection      Integrated
8      0      0      2549
9      763      0
10     L2_Cache      Manufacturer      Max_Power
11     Memory      Memory_Bandwidth
12     0      0      0
13     0      0      Memory_Speed      Memory_Type
14     Memory_Bus      Notebook_GPU
15     Name
16     0      0      0
17     Open_GL      PSU      Pixel_Rate      Power
18     _Connector      Process
19     40      0      0
20     0      0      Release_Price
21     ROPs      Release_Date
22     Resolution_WxH      SLI_Crossfire
23     0      0      0
24     0      0      Texture_Rate      VGA_
25     Shader      TMUs
26     Connection
27     107      538      0
28     758
```

BƯỚC 2: CHỌN LỌC CỘT VÀ ÉP KIỂU DỮ LIỆU

```
1 df <- df %>%
2   mutate(
3     # Gop ATI vao AMD truoc khi thuc hien cac buoc khac
```

```

4   manufacturer = ifelse(str_trim(manufacturer) == "ATI", "AMD", str_
trim(manufacturer)),
5   release_price = as.numeric(str_extract(release_price, "\\d+\\.?\\d*
")),
6   tdp = as.numeric(str_extract(tdp, "\\d+\\.?\\d*")),
7   memory_size = as.numeric(str_extract(memory_size, "\\d+\\.?\\d*")),
8   memory_bus = as.numeric(str_extract(memory_bus, "\\d+\\.?\\d*")),
9   core_speed = as.numeric(str_extract(core_speed, "\\d+\\.?\\d*")),
10  release_date = str_trim(release_date),
11  release_year = as.integer(format(as.Date(release_date, format="%d-%
b-%Y"), "%Y"))
12 )

```

Gộp ATI vào AMD: Sử dụng ifelse để sát nhập các dòng của hãng ATI vào AMD vì thực tế AMD đã mua lại ATI, giúp tập hợp dữ liệu hãng sản xuất một cách nhất quán.

str_extract(..., "\\d+\\.?\\d*"): Lọc lấy con số từ văn bản. Ví dụ: Từ chuỗi "738 MHz", R sẽ trích xuất đúng số "738" và hàm as.numeric sẽ biến nó thành số thực để tính toán.

as.Date & format: Chuyển chuỗi "01-Mar-2009" sang định dạng ngày chuẩn, sau đó chỉ lấy đúng 4 chữ số của năm (release_year). Theo lý thuyết, năm phát hành là yếu tố then chốt phản ánh xu hướng công nghệ theo thời gian.

Kết quả kiểm tra cấu trúc qua lệnh str(df) cho thấy quy trình ép kiểu tại Bước 2 đã hoàn tất thành công.

```

1 > str(df)
2 'data.frame':   3406 obs. of  8 variables:
3 $ release_price: num  NA NA NA NA NA NA NA NA NA NA ...
4 $ tdp           : num   141 215 200 NA 45 50 190 150 150 32 ...
5 $ memory_size  : num   1024 512 512 256 256 ...
6 $ memory_bus   : num    256 512 256 128 128 128 256 256 256 64 ...
7 $ core_speed   : num    738 NA NA NA NA NA 870 NA NA NA ...
8 $ manufacturer : chr    "Nvidia" "AMD" "AMD" "AMD" ...
9 $ memory_type  : chr    "GDDR3" "GDDR3" "GDDR3" "GDDR4" ...
10 $ release_year : int    2009 2007 2007 2007 2007 2007 2009 2007 2014
    2001 ...

```

BUỐC 3: XỬ LÝ NGOẠI LỆ

```

1 > count_outliers_by_raw_bounds <- function(raw_vec, clean_vec) {
2 +   raw_vec <- na.omit(raw_vec)
3 +   q1 <- quantile(raw_vec, 0.25)
4 +   q3 <- quantile(raw_vec, 0.75)
5 +   iqr <- q3 - q1
6 +   upper_bound <- q3 + 1.5 * iqr
7 +
8 +   # Dem xem trong tap sach co bao nhieu gia tri vuot nguong cua tap
   tho
9 +   return(sum(clean_vec > upper_bound, na.rm = TRUE))
10 + }

```

raw_vec <- na.omit(raw_vec): Loại bỏ tất cả các giá trị khuyết (NA) khỏi dữ liệu thô để việc tính toán phân vị không bị lỗi.

q1 <- quantile(raw_vec, 0.25): Tìm giá trị tại vị trí 25% của dữ liệu (Tứ phân vị thứ nhất).

q3 <- quantile(raw_vec, 0.75): Tìm giá trị tại vị trí 75% của dữ liệu (Tứ phân vị thứ ba).

iqr <- q3 - q1: Tính khoảng cách giữa Q3 và Q1. Đây là độ trải rộng của 50% dữ liệu nằm ở giữa.

upper_bound <- q3 + 1.5 * iqr: Xác định ranh giới trên. Theo lý thuyết thống kê của Tukey, bất kỳ giá trị nào lớn hơn ngưỡng này đều được coi là ngoại lệ (cực trị phía trên).

return(sum(clean_vec > upper_bound, na.rm = TRUE)): Kiểm tra xem trong tập dữ liệu đã làm sạch (clean_vec), có bao nhiêu giá trị vượt qua ngưỡng trên đã tính và trả về tổng số lượng đó.

Thống kê ngoại lệ trên console:

```
1 --- THONG KE NGOAI LE (Dua tren ranh gioi tap tho N=3406) ---
2 > cat("So luong ngoai le release_price trong mau phan tich:",
3 +     count_outliers_by_raw_bounds(df$release_price, df_clean$release_
4     price), "\n")
5 So luong ngoai le release_price trong mau phan tich: 33
6 >
7 > cat("So luong ngoai le tdp trong mau phan tich:",
8 +     count_outliers_by_raw_bounds(df$tdp, df_clean$tdp), "\n")
9 So luong ngoai le tdp trong mau phan tich: 23
10 >
11 > cat("So luong ngoai le core_speed trong mau phan tich:",
12 +     count_outliers_by_raw_bounds(df$core_speed, df_clean$core_speed)
13 , "\n")
14 So luong ngoai le core_speed trong mau phan tich: 43
```

BƯỚC 4: XỬ LÝ GIÁ TRỊ KHUYẾT

```
1 > df_clean <- df_clean %>%
2 +   mutate(
3 +     # Dien NA bang Median cho bien so lien tuc
4 +     tdp = ifelse(is.na(tdp), median(tdp, na.rm = TRUE), tdp),
5 +     memory_size = ifelse(is.na(memory_size), median(memory_size, na.
6 +     rm = TRUE), memory_size),
7 +     memory_bus = ifelse(is.na(memory_bus), median(memory_bus, na.rm =
8 +     TRUE), memory_bus),
9 +     core_speed = ifelse(is.na(core_speed), median(core_speed, na.rm =
10 +    TRUE), core_speed),
11 +     release_year = ifelse(is.na(release_year), median(release_year,
12 +     na.rm = TRUE), release_year),
13 +
14 +     # Dien NA bang Mode cho bien phan loai
15 +     memory_type = ifelse(is.na(memory_type) | memory_type == "", get_
16 +     mode(memory_type), memory_type),
17 +     manufacturer = ifelse(is.na(manufacturer) | manufacturer == "",
18 +     get_mode(manufacturer), manufacturer)
19 +   )
```

Lọc điều kiện (Filter):

- Loại bỏ các dòng thiếu release_price vì đây là biến mục tiêu Y, không thể tự ý điền vào được.

- Loại bỏ "Intel" vì toàn bộ GPU Intel trong tập dữ liệu thô đều không có giá, giữ lại sẽ gây nhiễu.

Điền khuyết bằng Trung vị (Median): Áp dụng cho các biến số (tdp, core_speed...). Trung vị được ưu tiên vì nó không bị kéo lệch bởi các card đồ họa siêu đắt tiền (ngoại lệ đã tìm thấy ở Bước 3).

Điền khuyết bằng yếu vị (Mode): Áp dụng cho các biến chữ (memory_type, manufacturer) bằng cách lấy giá trị xuất hiện nhiều nhất.

Sau quy trình điền khuyết theo phương pháp Median và Mode, bộ dữ liệu đã hoàn toàn sạch (0% NA), sẵn sàng cho việc xây dựng mô hình.

```
1 > print(colSums(is.na(df_clean)))
2 release_price      tdp memory_size memory_bus core_speed manufacturer
   memory_type release_year
3           0           0           0           0           0           0
               0           0
```

BƯỚC 5: MÃ HÓA BIẾN PHÂN LOẠI

```
1 df_clean <- df_clean %>%
2   mutate(
3     manufacturer = as.factor(manufacturer),
4     memory_type = as.factor(memory_type)
5   )
```

as.factor(): Hàm này chuyển đổi một cột dạng văn bản (character) thành dạng Yếu tố (Factor).

Biến định danh như nhà sản xuất và loại bộ nhớ đã được chuyển sang dạng Factor, giúp mô hình hồi quy có thể định lượng hóa tác động của thương hiệu đến giá phát hành.

```
1 $ manufacturer : Factor w/ 2 levels "1","2": 2 2 2 2 2 2 2 2 2 ...
2 $ memory_type  : Factor w/ 6 levels "DDR3","GDDR3",...: 4 4 3 3 3 3 3 3
   6 ...
```

BƯỚC 6: CHUẨN HÓA DỮ LIỆU (Z-SCORE NORMALIZATION)

```
1 df_final <- df_clean %>%
2   mutate(
3     tdp = scale(tdp)[,1],
4     core_speed = scale(core_speed)[,1],
5     memory_size = scale(memory_size)[,1],
6     memory_bus = scale(memory_bus)[,1]
7   )
```

Hàm scale mặc định trả về một ma trận, thêm [,1] để ép nó về dạng cột (vector) giúp dữ liệu đồng nhất.

Lệnh chạy: print(stats_check)

```
1 > print(stats_check)
2 Mean_TDP SD_TDP Mean_Core SD_Core
3 1         0         1         0         1
```

Kết quả trung bình bằng 0 và độ lệch chuẩn bằng 1 là bằng chứng đánh thép chứng minh dữ liệu đã được chuẩn hóa thành công.

BƯỚC 7.1: MÔ HÌNH HỒI QUY TUYẾN TÍNH BỘI

7.1.1 Kiểm định đa cộng tuyến (VIF)

```
1 mlr_model_initial <- lm(release_price ~ tdp + memory_size + memory_bus
2 +
3 core_speed + manufacturer + memory_type +
4 release_year, data = df_final)
5
6 # Su dụng chuẩn VIF < 5 thay vì VIF < 10 theo dung lý thuyết
7 vif_initial <- vif(mlr_model_initial)
8 print(vif_initial)
9
10 > print(vif_initial)
11
12          GVIF Df GVIF^(1/(2*Df))
13 tdp          2.140665 1          1.463101
14 memory_size  2.343250 1          1.530768
15 memory_bus   3.662332 1          1.913722
16 core_speed   3.238311 1          1.799531
17 manufacturer 1.617125 1          1.271662
18 memory_type  8.129892 5          1.233129
19 release_year 3.598408 1          1.896947
```

memory_type có VIF khoảng 8.12. Đây là bằng chứng để nhóm loại bỏ biến này ra khỏi mô hình chính thức nhằm đảm bảo tính khách quan.

7.1.2 Xây dựng mô hình chính thức (Log-Linear)

```
1 log_model_final <- lm(log(release_price) ~ tdp + memory_size + memory_
2 bus +
3 core_speed + manufacturer + release_year,
4 data = df_final)
5
6 summary(log_model_final)
```

Nhóm sử dụng hàm `log(release_price)` thay vì giá gốc. Việc lấy logarit giúp nén các giá trị cực trị (card đồ họa siêu đắt) và giúp sai số của mô hình tuân theo phân phối chuẩn tốt hơn.

Kết quả trên console:

```
1 Coefficients:
2
3      Estimate Std. Error t value Pr(>|t|)
4 (Intercept)  84.45346    22.49258   3.755 0.000192 ***
5 tdp          0.45471     0.01983  22.933 < 2e-16 ***
6 memory_size  0.17287     0.02134   8.101 3.54e-15 ***
7 memory_bus   0.07061     0.01532   4.610 5.01e-06 ***
8 core_speed   0.12173     0.02362   5.153 3.59e-07 ***
9 manufacturer2 0.37345     0.03610  10.345 < 2e-16 ***
10 release_year -0.03924     0.01116  -3.517 0.000473 ***
11 ---
12 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
13
14 Residual standard error: 0.3388 on 549 degrees of freedom
15 Multiple R-squared:  0.7854, Adjusted R-squared:  0.783
```

7.1.3 Kiểm định giả định mô hình

```
1 print(vif(log_model_final))
2
3 cat("\n--- 3b. KIEM DINH PHUONG SAI SAI SO (Breusch-Pagan) ---\n")
4 print(bptest(log_model_final))
5
6 cat("\n--- 3c. KIEM DINH PHAN PHOI CHUAN PHAN DU (Shapiro-Wilk) ---\n")
7 # Kiem tra xem log() da cai thien phan du chua
8 print(shapiro.test(residuals(log_model_final)))
```

Breusch-Pagan: Kiểm tra xem sai số của mô hình có ổn định không.

Shapiro-Wilk: Kiểm tra xem phần dư có phân phối chuẩn không.

Kết quả trên console:

```
1 > print(bptest(log_model_final))
2
3      studentized Breusch-Pagan test
4
5 data:  log_model_final
6 BP = 99.042, df = 6, p-value < 2.2e-16
7
8 >
9 > cat("\n--- 3c. KIEM DINH PHAN PHOI CHUAN PHAN DU (Shapiro-Wilk) ---\n")
10
11 --- 3c. KIEM DINH PHAN PHOI CHUAN PHAN DU (Shapiro-Wilk) ---
12 > # Kiem tra xem log() da cai thien phan du chua
13 > print(shapiro.test(residuals(log_model_final)))
14
15      Shapiro-Wilk normality test
16
17 data:  residuals(log_model_final)
18 W = 0.94765, p-value = 4.056e-13
```

7.1.4 Vẽ biểu đồ so sánh giá thực tế với giá dự đoán

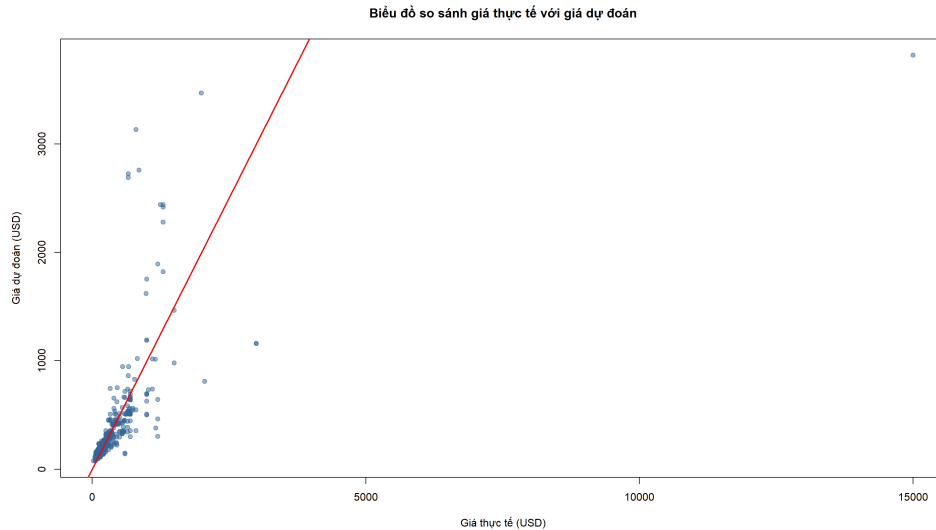
```
1 # Ve scatter plot
2 plot(actual_values, predicted_values,
3      main="Bieu do so sanh gia thuc te voi gia du doan",
4      xlab="Gia thuc te (USD)",
5      ylab="Gia du doan (USD)",
6      pch=19,
7      col=rgb(0.2, 0.4, 0.6, 0.5))
8 # Ke duong y=x de lam tham chieu ly tuong[cite: 4]
9 abline(0, 1, col="red", lwd=2)
```

Biểu đồ so sánh giá thực tế với giá dự đoán:

7.2. PHÂN TÍCH PHƯƠNG SAI

```
1 print(leveneTest(release_price ~ manufacturer, data = df_final))
```

Kiểm tra xem độ phân tán (biến thiên) của giá GPU thuộc hãng NVIDIA có bằng với độ phân tán của giá GPU hãng AMD hay không.



Hình 1: Biểu đồ so sánh giá thực tế với giá dự đoán

```
1 > print(levTest(release_price ~ manufacturer, data = df_final))
2 Levene's Test for Homogeneity of Variance (center = median)
3       Df F value    Pr(>F)
4 group  1  11.168 0.0008884 ***
5      554
```

```
1 anova_result <- oneway.test(release_price ~ manufacturer, data = df_
  final, var.equal = FALSE)
2 print(anova_result)
```

oneway.test(...): Hàm thực hiện phân tích phương sai một yếu tố.

var.equal = FALSE: Đây là tham số quan trọng nhất. Nó ra lệnh cho R thực hiện hiệu chỉnh Welch nhằm xử lý tình trạng phương sai không đồng nhất đã phát hiện ở bước trên.

Kết quả chạy trên console:

```
1 > print(anova_result)
2
3 One-way analysis of means (not assuming equal variances)
4
5 data: release_price and manufacturer
6 F = 18.39, num df = 1.00, denom df = 309.05, p-value = 2.409e-05
```